
Auto-Updating, Hallucination-Aware RAG System with Evaluation Dashboard

Full Project Roadmap & Milestone Guide

Production-Level AI Engineering Project

Milestones	6 clearly scoped stages
Estimated time	2–3 weeks at ~3–4 hrs/day
Difficulty	Intermediate Python → Advanced AI systems
Stack	Python 3.13 · FastAPI · LangChain · ChromaDB · OpenAI · RAGAS · Streamlit
Status	Living document — updated after each milestone completion

Milestone Completion Tracker

1	Project Setup & Foundation	■ Not started	
2	Document Ingestion Pipeline	■ Not started	
3	Retrieval Engine	■ Not started	
4	Answer Generation & Auto-Updating	■ Not started	
5	Hallucination Detection Layer	■ Not started	
6	Evaluation Dashboard	■ Not started	

System Architecture Overview

This project builds a production-style Retrieval-Augmented Generation (RAG) system. RAG is a technique where, instead of asking an AI model to answer from memory alone, you first search a database of your own documents for relevant information, then hand that information to the AI as context for its answer. This makes the AI more accurate, up-to-date, and trustworthy.

The five core data flows in the system:

1. Ingest	Documents (PDFs, web pages, text files) are loaded, split into chunks, converted to vectors (embeddings), and stored in ChromaDB.
2. Retrieve	A user question is also converted to a vector. The system finds the most similar chunks using semantic + keyword search, then reranks them.
3. Generate	The top chunks + the question are assembled into a prompt and sent to GPT-4. The answer streams back to the user with source citations.
4. Detect	Every answer is automatically scored for hallucination risk using RAGAS faithfulness scoring and an NLI model.
5. Evaluate	All queries, answers, retrieved chunks, and scores are logged. A live dashboard visualises system health over time.

Tech Stack Decision Guide

Python 3.13	Modern, fast, and the industry standard for AI engineering work.
FastAPI	High-performance async web framework. Used at Uber, Netflix, Microsoft. Much faster than Flask.
LangChain	The most popular framework for building LLM-powered apps. Handles loaders, splitters, chains, and memory.
ChromaDB	Free, open-source vector database that runs locally. No account required to get started.
OpenAI API	GPT-4 for generation, text-embedding-3-small for embeddings. Industry-standard quality.
RAGAS	The leading open-source RAG evaluation framework. Measures faithfulness, relevance, and precision.
APScheduler	Background job scheduler built into Python. Used for auto-updating the document index.
Streamlit	Turns a Python script into a web dashboard with almost no frontend code needed.
SQLite	Lightweight database for logging every query, answer, and evaluation score.

Project Setup & Foundation

MILESTONE 1 *Goal: A running skeleton app you can extend in every future milestone*

Before building anything intelligent, you need a solid, clean workspace. Think of this like setting up your kitchen before cooking — you need the right tools in the right places, a consistent structure, and clear labels on everything. This milestone sets up the FastAPI server (your app's front door), a clean folder structure, and a central config system so all future milestones can share settings safely.

What you will build

- A Python virtual environment isolating your project's dependencies
- A FastAPI web server with a /health endpoint to confirm the app is running
- A central config.py that reads from a .env file (API keys, model names, DB paths)
- The full folder structure all future milestones will plug into
- A requirements.txt with pinned versions so the project is reproducible
- A basic README.md documenting how to run the project

Key concepts

FastAPI	A Python web framework for building APIs. When a user sends a question, FastAPI receives it, routes it to the right code, and sends back the answer.
Virtual environment	An isolated Python sandbox. Your project's libraries live here and don't interfere with other Python projects on your machine.
.env file	A file storing secrets (API keys, passwords) that never gets committed to GitHub. The python-dotenv library reads it automatically.
Pydantic	A library that validates your data. Define what shape your data should be, and Pydantic will enforce it — catching bugs early.
Uvicorn	A lightning-fast web server that runs your FastAPI app. You start it from the terminal and it keeps the app alive.

Tools & libraries

Python 3.13 · FastAPI · Uvicorn · Pydantic v2 · python-dotenv · pytest

How it connects to the system

Every other milestone plugs into this foundation. The FastAPI server is the spine of the whole system. Ingestion, retrieval, generation, and evaluation all register as routes (endpoints) on this server. The config system means you change a model name or API key in one place and every part of the app picks it up automatically.

Completion checklist

- Running `uvicorn app.main:app --reload` starts without errors
- GET /health returns `{"status": "ok"}`

- The folder structure matches the project layout
- python-dotenv loads your .env file and config.py exposes all settings
- pytest tests/test_health.py passes

Document Ingestion Pipeline

MILESTONE 2 *Goal: Raw documents (PDF, web, txt) → searchable vector chunks in ChromaDB*

Imagine you have a library of textbooks, articles, and web pages. You want the AI to be able to answer questions about them. But the AI can't read your hard drive — you need to pre-process all those documents into a format it can search quickly. That format is called an embedding (a list of numbers representing meaning). This milestone builds the pipeline that takes raw documents and turns them into searchable chunks.

What you will build

- Document loaders: read PDF files, .txt files, and web URLs
- A text splitter that breaks documents into ~500-word chunks with 50-word overlap
- An embedding function using OpenAI's text-embedding-3-small model
- A ChromaDB client that stores chunks + their vectors + metadata (source, page number)
- A POST /ingest endpoint that accepts a file or URL and runs the full pipeline
- Duplicate detection (don't re-ingest a document that is already in the database)
- Ingestion logging (record what was ingested, when, how many chunks)

Key concepts

Chunking	Cutting a long document into smaller pieces — like index cards from a textbook. Each chunk is self-contained enough to answer a question, but small enough to search fast. Overlap ensures context isn't lost at the boundaries.
Embeddings	A mathematical representation of text as a list of ~1500 numbers. Similar meanings produce similar number patterns. 'Dog' and 'puppy' are close. 'Dog' and 'spaceship' are far apart. This is how semantic search works.
Vector database	A special database optimised to find vectors (lists of numbers) that are similar to each other. ChromaDB stores your chunks + their embeddings and can search them in milliseconds.
Metadata	Extra data stored alongside each chunk: source filename, page number, ingestion date. Critical for citations and debugging.
Idempotency	Running the ingest twice should not create duplicates. You achieve this by hashing the document content and checking before inserting.

Tools & libraries

LangChain (document loaders, text splitters) · OpenAI Embeddings API · ChromaDB · PyPDF2 · BeautifulSoup4 (web scraping)

How it connects to the system

This milestone feeds the retrieval engine in Milestone 3. Without ingested, embedded documents, there is nothing to search. The metadata stored here powers the citations shown in Milestone 4, and the ingestion logs are displayed on the dashboard in Milestone 6.

Completion checklist

■	POST /ingest with a PDF file path processes and stores chunks in ChromaDB
■	ChromaDB collection shows the correct number of chunks after ingestion
■	Re-ingesting the same document does not create duplicates
■	Each chunk has metadata: source, page, chunk_id, ingested_at
■	pytest tests/test_ingestion.py passes

Retrieval Engine

MILESTONE 3 *Goal: Given a question, find the most relevant chunks from the database*

Now your documents are stored. When a user asks 'What are the side effects of aspirin?', you need to instantly find the handful of chunks from your database that best answer that question. This milestone builds a smart, multi-stage search system — first casting a wide net (semantic search), then refining with keyword matching, then reranking the survivors to pick only the best context for the AI.

What you will build

- Semantic search: convert the question to an embedding, search ChromaDB for similar chunks
- BM25 keyword search: traditional word-match search as a complement to semantic search
- Hybrid search: combine results from both methods with a weighted score
- A reranker that takes the top 20 results and selects the best 5
- A context assembler that packages the top chunks into a clean prompt-ready string
- A GET /retrieve endpoint that returns the top chunks for a given question
- Retrieval metadata: return relevance scores, source info, and chunk content

Key concepts

Semantic search	Searches by meaning, not exact words. If you ask 'how do dogs play?', it finds chunks about 'puppies frolicking' because they are semantically similar, even with no word overlap.
BM25 keyword search	The classic search algorithm — finds chunks containing the exact words from the query. Works well for product names, codes, or proper nouns that semantic search might miss.
Hybrid search	Combines semantic and keyword search results. Most production RAG systems use both because neither alone handles all query types well. Results are merged using Reciprocal Rank Fusion (RRF).
Reranking	A second, slower but more accurate AI model that re-reads your top candidate chunks and re-scores them. You only run it on the top 20 to keep latency low, but it significantly improves the final top 5.
Cosine similarity	The mathematical measure used to compare two embeddings. A score of 1.0 = identical meaning. A score of 0.0 = completely unrelated. ChromaDB uses this internally.

Tools & libraries

ChromaDB (semantic search) · rank-bm25 (keyword search) · Cohere Rerank API or HuggingFace cross-encoder/ms-marco-MiniLM-L-6-v2 · NumPy

How it connects to the system

The retrieval engine is the bridge between your stored documents and the AI's answer. Better retrieval = more accurate answers = lower hallucination risk. The chunks returned here are passed directly to the generation chain in Milestone 4, and the retrieval quality is measured by Milestone 5 (RAGAS context

precision score).

Completion checklist

■	GET /retrieve?q=your+question returns the top 5 most relevant chunks
■	Results include relevance score, source, and chunk text
■	Hybrid search consistently outperforms semantic-only on a test question set
■	Reranker is applied and improves result ordering
■	pytest tests/test_retrieval.py passes

Answer Generation & Auto-Updating

MILESTONE 4 *Goal: Ask a question → get a cited, streamed, up-to-date answer in real time*

Now you have the right chunks. This milestone assembles them into a carefully crafted prompt for GPT-4 and streams the answer back to the user word by word — just like ChatGPT. The prompt is engineered to keep the AI grounded: it should only answer from the provided context, never from its own imagination. Every answer includes citations. And a background scheduler automatically re-ingests documents on a schedule so the knowledge base stays fresh.

What you will build

- A grounded prompt template that instructs GPT-4 to answer ONLY from provided context
- A LangChain generation chain that combines retrieval + generation in one call
- Streaming responses using FastAPI's StreamingResponse (words appear as they generate)
- Source citations: every answer includes the document names and page numbers it drew from
- A POST /ask endpoint that accepts a question and returns a streamed, cited answer
- An APScheduler background job that re-ingests documents every N hours
- A manual POST /refresh endpoint to trigger re-ingestion on demand

Key concepts

Prompt engineering	The art of writing instructions to the AI that produce reliable, grounded answers. Your prompt says: 'Answer only from this context. If the answer is not here, say you do not know.' This is what separates RAG from plain ChatGPT.
Grounding	Forcing the AI to use only the retrieved context rather than its general training knowledge. This dramatically reduces hallucination risk and keeps answers relevant to your specific documents.
Streaming	Instead of waiting for the entire answer before showing anything, you send words to the user as soon as GPT-4 generates them. The user sees output immediately, making the app feel fast even for long answers.
RAG chain	A LangChain pattern that wires together retrieval + prompt assembly + LLM call into a single reusable object. Calling chain.invoke({question}) does all three steps automatically.
APScheduler	A Python library that runs functions on a schedule (every hour, every day at 2am, etc.) in a background thread — without blocking the web server.

Tools & libraries

OpenAI Chat API (GPT-4o) · LangChain LCEL (LangChain Expression Language) · FastAPI StreamingResponse · APScheduler · tiktoken (token counting)

How it connects to the system

This is the user-facing heart of the system. It is where Milestones 2 (chunks) and 3 (retrieval) come together to produce actual answers. The answers generated here are then scored by Milestone 5 (hallucination detection) and logged for Milestone 6 (the evaluation dashboard). The auto-update loop

ensures the system stays current without manual intervention.

Completion checklist

■	POST /ask?q=your+question returns a streamed response visible word-by-word
■	The answer includes source citations (document name + page)
■	Asking about something NOT in the documents returns 'I don't know' — not a made-up answer
■	APScheduler logs show the re-ingestion job running on schedule
■	pytest tests/test_generation.py passes

Hallucination Detection Layer

MILESTONE 5 *Goal: Automatically flag answers not supported by retrieved context*

Even a well-grounded RAG system can sometimes produce answers that go beyond what the context actually says. This is called hallucination. In a medical, legal, or financial application, an undetected hallucination could be dangerous. This milestone wraps every generated answer in an automatic quality check — scoring it for faithfulness before it reaches the user.

What you will build

- RAGAS faithfulness scorer: measures 0-1 how well the answer sticks to the retrieved context
- NLI (Natural Language Inference) sentence-level checker: classifies each sentence as entailed, contradicted, or neutral relative to the context
- Self-consistency checker: ask GPT-4 the same question 3 times, flag if answers disagree significantly
- A confidence score attached to every response (high / medium / low / unverified)
- A /score endpoint that accepts an answer + context and returns hallucination metrics
- Integration with the generation pipeline: every answer from /ask is auto-scored before returning
- A warning label shown to the user when confidence is below threshold

Key concepts

Hallucination	When an AI generates text that sounds correct but is not supported by the provided source material. Common in all LLMs. The goal is detection and flagging, not elimination.
Faithfulness	A RAGAS metric (0–1) measuring whether every claim in the generated answer is grounded in the retrieved chunks. Score of 1.0 = everything the AI said is traceable to context. Score below 0.5 = likely hallucination.
NLI (Natural Language Inference)	A classification task: given a premise (your context chunk) and a hypothesis (a sentence from the answer), predict whether the premise entails, contradicts, or is neutral to the hypothesis. Used as a sentence-level fact-checker.
Self-consistency	Generate the same answer multiple times with randomness (temperature > 0) and measure agreement. High variance in answers = low confidence. This reveals when GPT-4 is unsure about something.
RAGAS triad	The three core RAG metrics: (1) Context Precision — did you retrieve the right chunks? (2) Faithfulness — does the answer stick to the context? (3) Answer Relevance — does the answer address the question?

Tools & libraries

RAGAS · HuggingFace transformers (cross-encoder/nli-deberta-v3-small) · sentence-transformers · OpenAI API (self-consistency sampling)

How it connects to the system

This milestone is what makes the system 'hallucination-aware' — the key differentiator in the project name. Every answer from Milestone 4 passes through this layer. The confidence scores it produces are stored in

the query log (used by Milestone 6) and returned to the user alongside every answer. Production AI systems at companies like Perplexity and Cohere use similar pipelines.

Completion checklist

■	Every answer from /ask includes a hallucination_score and confidence_level field
■	A fabricated answer (manually crafted to contradict context) scores below 0.4
■	A faithful answer (directly drawn from context) scores above 0.8
■	NLI checker correctly flags contradicted sentences
■	pytest tests/test_hallucination.py passes

Evaluation Dashboard

MILESTONE 6 *Goal: A visual dashboard showing how well your system is performing in real time*

You now have a fully functional, hallucination-aware RAG system. But how do you know if it is getting better or worse over time? How do you catch regressions (things breaking silently) before users notice? This milestone builds the observability layer — a live dashboard that every senior AI engineer would expect to see in a production system.

What you will build

- A SQLite query log database: record every question, answer, retrieved chunks, scores, and latency
- A golden dataset: 20 hand-crafted question/answer pairs used to benchmark system quality
- Automated benchmark runner: runs golden dataset nightly and stores results
- A Streamlit dashboard showing: average faithfulness score, hallucination rate (%), retrieval latency distribution, top failed queries, document freshness, RAG triad over time
- A GET /metrics endpoint returning current system health as JSON (for API consumers)
- Alerting: if average faithfulness drops below threshold, log a critical warning

Key concepts

Golden dataset	A hand-crafted set of questions with known correct answers, used to measure whether system changes make performance better or worse. Like an exam for your AI — you run it after every change.
Regression	When a code change accidentally makes performance worse on a metric you were already passing. The golden dataset + automated runner catches this automatically.
RAG triad	Three metrics measured by RAGAS: Context Precision (did you retrieve the right chunks?), Faithfulness (does the answer stick to context?), and Answer Relevance (does the answer address the question?). All three should be high.
Latency percentiles	Instead of average latency, production systems track p50, p95, and p99 latency. p95 = 95% of requests are faster than this. A healthy system has p95 latency under 3 seconds.
Streamlit	A Python library that turns a Python script into a web dashboard. Define your charts and tables in Python, run <code>streamlit run dashboard.py</code> , and a browser tab opens with a live UI. No HTML/CSS needed.

Tools & libraries

SQLite + SQLAlchemy · RAGAS · Streamlit · Pandas · Plotly Express · APScheduler (benchmark runner)

How it connects to the system

The dashboard is the window into the entire system's health. It consumes logs from Milestone 2 (ingestion events), Milestone 3 (retrieval latency), Milestone 4 (answer latency, citations), and Milestone 5 (hallucination scores). Completing this milestone means you have a complete, observable, production-style AI system — exactly what you would be expected to build on an AI engineering team.

Completion checklist

■	streamlit run app/evaluation/dashboard.py launches a working dashboard
■	Dashboard shows real data from the query log (not mock data)
■	Golden dataset runner executes automatically via APScheduler
■	GET /metrics returns faithfulness, hallucination_rate, avg_latency_ms, total_queries
■	pytest tests/test_evaluation.py passes

Recommended Project Structure

Keep this folder structure from day one. Each folder maps to a milestone. New code always goes in the right place, making the project easy to navigate and test.

```
rag-system/  
■■■ app/  
■ ■■■ main.py # FastAPI app entry point  
■ ■■■ config.py # Central config (reads .env)  
■ ■■■ ingestion/ # Milestone 2  
■ ■ ■■■ loaders.py # PDF, web, txt loaders  
■ ■ ■■■ chunker.py # Text splitter logic  
■ ■ ■■■ embedder.py # Embedding + ChromaDB writes  
■ ■■■ retrieval/ # Milestone 3  
■ ■ ■■■ semantic.py # ChromaDB vector search  
■ ■ ■■■ keyword.py # BM25 search  
■ ■ ■■■ hybrid.py # Fusion + reranking  
■ ■ ■■■ context.py # Context assembler  
■ ■■■ generation/ # Milestone 4  
■ ■ ■■■ chain.py # LangChain RAG chain  
■ ■ ■■■ prompt.py # Prompt templates  
■ ■ ■■■ scheduler.py # APScheduler auto-update  
■ ■■■ hallucination/ # Milestone 5  
■ ■ ■■■ ragas_scorer.py # RAGAS faithfulness  
■ ■ ■■■ nli_checker.py # HuggingFace NLI  
■ ■ ■■■ consistency.py # Self-consistency check  
■ ■■■ evaluation/ # Milestone 6  
■ ■■■ logger.py # SQLite query logger  
■ ■■■ benchmark.py # Golden dataset runner  
■ ■■■ dashboard.py # Streamlit dashboard  
■■■ data/  
■ ■■■ documents/ # Your source documents  
■ ■■■ golden_dataset.json # Benchmark Q&A; pairs  
■■■ tests/  
■ ■■■ test_health.py  
■ ■■■ test_ingestion.py  
■ ■■■ test_retrieval.py  
■ ■■■ test_generation.py  
■ ■■■ test_hallucination.py  
■ ■■■ test_evaluation.py  
■■■ .env # Secret keys – NEVER commit
```

```
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

Production Best Practices Applied Throughout

Separation of concerns	Each module does exactly one job. Ingestion does not know about generation. Generation does not know about evaluation. This makes every piece testable and replaceable.
Config over hardcoding	No API keys, model names, or thresholds buried in code. Everything is in config.py, read from .env. Change behaviour without touching logic.
Async-first design	FastAPI endpoints use async/await so the server handles multiple requests simultaneously without freezing.
Structured logging	Every major action (ingest, retrieve, generate, score) is logged with timestamp, duration, and outcome. This is what makes the dashboard possible.
Graceful degradation	If hallucination scoring fails, the system still returns an answer but marks it as 'unverified' rather than crashing.
Reproducibility	requirements.txt pins exact library versions. Anyone cloning the repo gets an identical environment.
Test coverage	Each milestone ships with pytest tests. Tests run against a small in-memory ChromaDB so they are fast and do not require real API calls.